

The Interface Is a Variable: Measuring the Cost and Reliability of Purpose-Built UI Representations for LLM Agents

Abstract

Language-model agents usually operate software through interfaces built for people. They read screenshots, accessibility trees, or raw HTML, and they act by clicking and typing. This paper treats the interface representation, the form in which an application shows its state and its available actions to an agent, as an experimental variable rather than as a fixed cost of automation. We hold one small online store (MiniShop), its ten tasks, and a grader that inspects backend order records constant, and present the same store to the same model in four ways: a screenshot (C1), an accessibility tree (C2), flat tools (C3), and a purpose-built JSON view document (C4) that states which screen the agent is on and which actions are allowed right now. We measure task success, token cost, step count, and illegal actions, and we read the results as a cost-reliability frontier: a representation earns its place only if nothing else is both cheaper and at least as reliable. On the first model measured, both surfaces built for people lose outright. The screenshot (C1) costs roughly fifteen times the tokens of the cheapest condition and completes fewer than a third of its tasks. The accessibility tree (C2) is cheaper than pixels but error-prone. Between the structured conditions the verdict is closer: the view document (C4) finishes tasks in fewer steps, but it spends more tokens than flat tools (C3) and does not improve success, so flat tools (C3) is preferred on this store. All numbers come from one general model (`gemini-2.5-flash`) on one synthetic store. We present them as a measurement method and a first data point, and we identify the conditions under which a purpose-built agent surface should begin to pay off.

1. Introduction

An agent that operates software runs a loop. It looks at the application, decides on one action, performs that action, and looks again. Every look enters the model as input tokens, and every decision is billed as output tokens. Across the loop the application itself does not change. The only thing that changes is the form in which the agent sees it.

That form is almost always inherited from software built for people. Three channels dominate practice today, and each fails the agent in its own way.

1. **Screenshot (C1).** The model sees pixels and must translate intent into click coordinates. Mapping words like "the Add to cart button" onto the right pixels is called grounding, and

it is a well-documented failure mode of GUI agents (SeeClick, UI-TARS, Claude computer use). Worse, image cost tracks the size of the viewport rather than the amount of information on the page, so a blank page and a busy page at the same window size cost exactly the same.

2. **Accessibility tree (C2).** The model reads the text tree that assistive technology uses. The tree is verbose. Mind2Web filters it because it is too large to send whole, and UIFormer measures interface text at 80 to 99 percent of an agent's total token cost. The tree also names controls that the backend will reject, which turns confident clicks into errors.
3. **Flat tools (C3).** The model receives a catalog of function schemas and calls them. After each call it gets back a one-line status reply that names the current view and echoes a field or two, and nothing else. This is close to how the Model Context Protocol exposes capabilities. It is cheap per call once the schema is paid for, but nothing describes what is on the screen or which actions are currently valid, so the model must work that out from its own history.

None of these channels was designed to answer, cheaply and unambiguously, the two questions an agent asks at every step: *what is true right now, and what may I do next?*

The alternative we measure is a **view document (C4)**. It is a short JSON object that the application generates from its own backend state every time the agent looks. The document names the screen the agent is on, lists what is on that screen, and lists every action the interface offers, each marked as available or not right now, together with a description of the arguments it takes. Nothing in it is guessed from pixels or scraped from a page. It comes from the same backend records that the grader checks, so what the agent reads and what the grader scores cannot quietly disagree. Section 4.1 shows a real one, captured mid-purchase.

Behind this sits a product question: should applications offer agents a surface like this, the way they already offer developers an API? This paper does not answer that question, and it does not propose a standard. It contributes the measurement the question needs. We hold the store, the tasks, the grader, and the model fixed, vary only the representation, and read the outcome on a cost-reliability frontier, where a representation survives only if nothing else is both cheaper and at least as reliable.

The experiment in one paragraph. One synthetic store (MiniShop), ten frozen tasks (seven that should succeed, three that should be refused), one deterministic grader that inspects backend order records, one model at a time, temperature zero, at most twenty steps per task. The agent's memory is the task text, its prior actions, and the current observation. The only thing that varies is how the agent sees and acts: screenshot (C1), accessibility tree (C2), flat tools (C3), or view document (C4). The two structured conditions change the store through the same seven operations, so any difference between them comes from how the store is described, not from a different set of buttons.

What we found, in brief. On the first model measured, the human surfaces lose decisively. The screenshot (C1) never completes a purchase task and costs about 46,000 input tokens per task. The accessibility tree (C2) succeeds more often but commits the most illegal and malformed actions. Flat tools (C3) completes every task. The view document (C4) nearly matches it, uses the fewest steps of any condition, costs more tokens than flat tools (C3), and fails exactly one task, because the model ignored an action that the document had clearly marked as unavailable. These numbers come from one model on one application. The plan is a grid of models crossed with applications, this is its first entry, and we treat the numbers accordingly.

Contributions.

1. A controlled method for comparing interface representations at *matched action grain*: flat tools (C3) and view document (C4) change the store through the same seven operations, so a win for the document cannot be explained by a smaller or simpler action set.
2. A cost-reliability reading with paired per-task statistics, rather than a success-only leaderboard: conditions are compared task by task, with confidence intervals computed by resampling (Section 5 explains the procedure).
3. First evidence on one store and one model, anchored by a model-free observation-cost baseline (what each representation costs to read before any model is involved) and a deterministic execution grader.

Research questions.

- **RQ1.** Does the representation change token cost, step count, task success, and illegal-action rate on a fixed task set?
- **RQ2.** Is some representation cheaper and at least as reliable as reusing the human page? In particular, does the view document (C4) dominate the screenshot (C1) and the accessibility tree (C2)?
- **RQ3.** Does the ordering of conditions hold across general models, or is it an artifact of one model?

The matching hypotheses: **H1**, cost and reliability differ across conditions. **H2**, the view document (C4) sits on the cost-reliability frontier and the screenshot (C1) is dominated. **H3**, the ordering of conditions is stable across general models even if the size of the gaps is not. On this first model, H1 holds at the coarse level (human surfaces against structured ones). H2 does not hold in full: the view document (C4) beats the human surfaces but not flat tools (C3). H3 is untested.

How the paper is organized. Section 2 collects the terms used throughout. Section 3 places the study among prior work. Section 4 describes the store, the conditions, the tasks, and a real view document (C4). Section 5 defines the metrics and the analysis. Section 6 reports results in three layers: a model-free baseline, an oracle control, and the model run. Section 7 separates what

these results show from what later measurements must show. Section 8 explains how the page and the document stay consistent, what that mechanism assumes about the application, and how a real application, including an existing one, could ship a view document (C4). Sections 9 and 10 cover limitations and future work.

2. Terms used in this paper

Conditions are always written as method name plus identifier.

- **Screenshot (C1).** A rendered image of the human page. The agent clicks coordinates, types, and scrolls.
- **Accessibility tree (C2).** The text tree of that page (roles and names). The agent clicks or fills named elements.
- **Flat tools (C3).** A catalog of callable functions. Each call returns a short status reply that names the current view and echoes the selected size and cart count. Nothing lists what is on the view or which actions are currently valid.
- **View document (C4).** A JSON object with `view`, `state`, `entities`, and `affordances`, generated from backend state.
- **Affordance.** The design term for one action an interface offers, such as `open_product` or `pay`. In the document each affordance carries an `enabled` flag and an argument schema.
- **Matched action grain.** Flat tools (C3) and view document (C4) change the store through the same seven operations, so differences between them come from how state is communicated, not from what the agent can do. Section 4 spells out the one read-only exception.
- **Illegal action.** An action the backend rejects.
- **Malformed action.** A model reply that is not a usable action at all (wrong shape). Counted separately from illegal actions.
- **Ignored affordance.** The model attempts an action the document marked `enabled: false`. Task t10 is the running example.
- **Step.** One observe, decide, act cycle. Runs are capped at 20 steps.
- **Execution-based grader.** A deterministic check over backend order records. Never another model.
- **Cost-reliability frontier.** Token cost plotted against success. A condition is *dominated* if another condition is no more expensive and at least as reliable.
- **Refusal task.** A task whose correct outcome is to decline (for example, the requested size is sold out). Reported separately, because doing nothing can pass one.

3. Related work

Most prior work asks which *agent* is better at operating a computer and treats the interface as a fixed property of the benchmark. This study inverts that. The model is held fixed and the interface is the variable.

Reusing the human page. Agents that read the DOM (the tree of elements a browser builds from a page's HTML) or the accessibility tree (Mind2Web, WebArena) correspond to our accessibility tree (C2). Screenshot and pixel systems (Claude computer use, UI-TARS, SeeClick) correspond to our screenshot (C1). In those lines of work, grounding is the central difficulty, and screenshots cost on the order of one to two thousand tokens each. Our question is not how to ground pixels better. It is whether a non-pixel surface, authored for the agent, is better for ordinary general models. Specialized pixel-native models are deferred to a later check.

Compressing the human tree. UIFormer and Prune4Web cut down and restructure the human DOM or accessibility tree to save tokens. UIFormer reports interface text at 80 to 99 percent of agent token cost and reductions around 49 to 56 percent. This is the closest neighboring line, and it improves the accessibility tree (C2). The view document (C4) differs in origin: it is generated from backend state, not derived from the human page, and we measure it as a controlled experimental arm rather than as a plug-in optimizer.

Agent-facing protocols. A2UI runs in the opposite direction, letting an agent generate interface for a human to view. The Model Context Protocol standardizes tool catalogs, which is close to flat tools (C3). The view document (C4) is application-to-agent, per view, and carries validity constraints, and we give it the same action set as flat tools (C3) so the document's added value can be isolated. We use it as a measurement probe, not as a proposed standard.

Benchmarks. WebArena, VisualWebArena, Mind2Web, WebShop, AndroidWorld, and MiniWoB++ vary the agent, fix the interface, and grade by execution. We reuse the execution-grading idea and invert the design. We also report a model-free observation-cost baseline, which capability benchmarks do not provide.

Table R1. Positioning against each strand (verified sources only).

Strand	Representative work	What it does	How this study differs
DOM / accessibility agents	Mind2Web; WebArena	Operate the human page from HTML or a tree	That channel is our accessibility tree (C2), one of four controlled conditions

Strand	Representative work	What it does	How this study differs
Screenshot / pixel computer use	Claude computer use; UI-TARS; SeeClick	Perceive pixels, emit coordinates	That channel is our screenshot (C1). We study representation, not better grounding
Optimizing the human tree	UIFormer; Prune4Web	Compress a human-derived tree	Closest neighbor. The view document (C4) is generated from backend state instead
Agent-UI protocols	A2UI; MCP	A2UI is agent to human; MCP is a tool catalog	The view document (C4) is application to agent and matched in grain to flat tools (C3)
GUI / web benchmarks	WebArena; VisualWebArena; Mind2Web; WebShop; AndroidWorld; MiniWoB++	Vary the agent, fix the interface	We fix the agent and vary the interface

What this study does not claim. That representation matters is already known: Mind2Web filters the DOM because it is too large, SeeClick is motivated by lengthy HTML, and UIFormer measures how much of an agent's budget the interface consumes. Execution grading is not new. Token-efficient interface trees are not new. Agent-facing surfaces are already discussed (A2UI, MCP). Masking invalid actions is a known idea in reinforcement learning and tool use, and we do not attach a citation we have not verified. The claim here is the controlled four-way comparison on a fixed application, model, and grader, and the scope of that claim is bounded by the setup: one store, ten tasks, one model, and a hand-authored view document (C4).

4. Study design

The store. MiniShop is a small shop with a catalog of eight products, product pages, a cart, and a checkout. Its backend state is the single source of truth for grading. The human pages carry realistic clutter (promotional copy, a search box, decorative filters, help links), so the screenshot (C1) and accessibility tree (C2) conditions see a page genuinely built for people rather than a stripped-down agent shell. Sold-out sizes are visible but disabled on the page, so a pixel or tree agent can still attempt them. The structured conditions act through a JSON API. The screenshot (C1) and accessibility tree (C2) conditions drive the human pages in a real browser (headless Chromium driven by Playwright, viewport 1280 by 800 pixels) and never touch that API to act.

Why only the interface varies. If the store, the tasks, the grader, or the model changed at the same time as the interface, no difference could be attributed to the interface. So everything else is pinned.

Table I1. Held fixed versus varied.

Held fixed	Varied
MiniShop, one backend, one grader	How the agent <i>sees</i> the store
Ten frozen tasks	How the agent <i>acts</i> on it
One model per comparison; temperature 0; step cap 20	Presence of the view document, comparing view document (C4) against flat tools (C3)
Memory: task, prior actions, current observation	

One difference we cannot remove (in experimental terms, a confound) remains. The screenshot (C1) and accessibility tree (C2) act through coordinates and named elements, which cannot be identical to the structured conditions' function calls. Comparisons between human and structured surfaces therefore carry an action-vocabulary difference along with the representation difference. The comparison that isolates the *document* itself is flat tools (C3) against view document (C4).

Matched action grain. Both structured conditions change the store through the same seven operations: `open_product`, `set_size`, `add_to_cart`, `go_catalog`, `go_checkout`, `set_address`, and `pay`. If the view document (C4) had fewer or simpler actions, any win could be explained as "fewer buttons," so the acting vocabulary is held equal. What differs is how each condition reads the store, which is exactly the variable under test. Flat tools (C3) reads through one extra read-only function, `list_products`, which returns the catalog, plus the status reply that follows each call, for example `{"status": "ok", "view": "product", "selected_size": "L", "cart_count": 1}`. It needs `list_products` because nothing else would ever tell it a product id. The view document (C4) has no `list_products` and does not need it: the document itself lists the products whenever the agent is on the catalog. There is also a mechanical difference in delivery. Flat tools (C3) uses the provider's native tool-calling interface, which re-sends the function schema with every request; the view document (C4) places the document in the prompt and the model replies with one JSON action. What this matching does not separate is *why* the document helps when it helps: because it is compact, or because it carries constraints. Those two mechanisms stay entangled until the ablation described in Section 6.5 is run. Models are never mixed within a comparison.

What the prompts say. Every condition runs with the same short set of shared rules (follow the task exactly, stop if the item cannot be purchased, the backend is the source of truth) followed by a few lines describing that condition's interface and how to use it. The usage lines cannot be identical across conditions, and they are not neutral: the flat tools (C3) prompt names a sensible order of operations (list the products, open one, set a size, and so on), and the view document (C4) prompt advises preferring enabled actions. Some part of any between-condition gap is

therefore instruction authorship rather than representation. This is a confound shared by every cross-interface comparison, we state it rather than pretend it away, and equalizing the guidance is a cheap ablation queued in Section 10.

The four conditions.

Condition	What the agent sees	How it acts
Screenshot (C1)	Rendered image of the human page	Click coordinates, type, scroll
Accessibility tree (C2)	Text tree of that page	Click or fill named elements
Flat tools (C3)	Function catalog and short status replies	Call those functions
View document (C4)	JSON: view, state, entities, affordances	Invoke an enabled affordance with typed arguments

Tasks and grading. Ten frozen tasks: seven purchases that should succeed and three requests that should be refused. The grader inspects backend order records and is never another model. Because every number in Section 6 rests on it, its criteria are stated here exactly as implemented. A purchase passes if the most recent order contains the requested product in the requested size and its shipping address contains the requested street line. A refusal passes if the forbidden product and size pair was never ordered (t03 and t07 ask for sold-out sizes) or if no order exists at all (t05 asks to pay with an empty cart). These checks are minimal, and the slack in them is real. An extra item in the order, an earlier wrong order, or the right street in the wrong city would all slip through, and an agent that "helpfully" substituted an in-stock size and bought it would pass a refusal task, which a real user would count as a serious failure. The recorded runs had no room to exploit this slack: the passing structured runs finished on the shortest correct path, refusal passes in the text conditions ended within two or three steps, and the screenshot (C1) never completed a purchase anywhere in the sweep, so no substitute order slipped in. The checks still deserve tightening before a model appears that tests them, and Section 10 queues that. Because an agent that does nothing can technically pass a refusal, refusal results are reported separately and never inflate the headline success rate.

Table T1. The frozen task set.

ID	Expect	Instruction (abridged)
t01	success	Navy Crew Tee, size M, to 18 Cedar Ave, Portland
t02	success	Harbor Blue Tee, size L, to 18 Cedar Ave, Portland

ID	Expect	Instruction (abridged)
t03	refusal	Harbor Blue Tee, size M (sold out)
t04	success	Studio Mug, to 9 Pine Street, Austin
t05	refusal	Pay with an empty cart
t06	success	Ash Hoodie, size S, to 4 Market Road, Seattle
t07	refusal	Signal Cap, size S (sold out)
t08	success	Field Notebook, to 100 King St, Boston
t09	success	Day Bottle and Crew Socks, to 18 Cedar Ave, Portland
t10	success	Navy Crew Tee, size L, to 77 Oak Lane, Denver

How a run works. One run is one combination of condition, task, and repeat. The agent loops (observe, act, observe) and the run ends when the model declares it is finished, when the grader marks a purchase task complete, or at the twenty-step cap. Ending purchase runs at the grader's signal keeps the sweep cheap, but it costs observability: the agent never has to notice on its own that it is done, and anything it might have done after succeeding is invisible. Section 9 returns to this. A full sweep is 4 conditions by 10 tasks by 5 repeats, 200 runs. Before any model runs, a deterministic oracle (a scripted agent that already knows the correct action sequence for every task) is executed on flat tools (C3) and view document (C4). It scores 100 percent on both, which establishes that any later miss belongs to the model, not to the harness or the grader. Every run writes an independent trace with per-step usage, so the analysis reads finished artifacts rather than live state, and conditions are launched separately so one crash cannot take down the others.

4.1 A real view document (C4)

The document always has four parts. `view` names the screen the agent is on. `state` holds the session fields that matter on that screen. `entities` are the objects shown. `affordances` are the actions, each with an `enabled` flag and an `input` schema for its arguments.

The listing below is an actual document from MiniShop, not a sketch. The agent has opened the Navy Crew Tee, selected size L, and added it to the cart. It is still on the product page. Checkout has not been reached, so entering a shipping address is not yet a legal move, and the document says so: `set_address` and `pay` carry `enabled: false`, while `go_checkout` is available. The product-id list is shortened here for print; the live document lists every catalog id.

```

{
  "view": "product",
  "state": {
    "product_id": "tee-navy",
    "selected_size": "L",
    "cart_count": 1,
    "address": "",
    "order_count": 0
  },
  "entities": {
    "product": {
      "id": "tee-navy",
      "name": "Navy Crew Tee",
      "price": 28,
      "sizes_in_stock": ["S", "M", "L"]
    },
    "cart": [
      {
        "product_id": "tee-navy",
        "name": "Navy Crew Tee",
        "size": "L",
        "price": 28
      }
    ]
  },
  "affordances": [
    {"id": "open_product", "enabled": true,
     "input": {"type": "object",
               "properties": {"product_id": {"type": "string"}},
               "required": ["product_id"]}},
    {"id": "set_size", "enabled": true,
     "input": {"type": "object",
               "properties": {"size": {"type": "string", "enum": ["S", "M", "L"]}},
               "required": ["size"]}},
    {"id": "add_to_cart", "enabled": true,
     "input": {"type": "object", "properties": {}}},
    {"id": "go_catalog", "enabled": true,
     "input": {"type": "object", "properties": {}}},
    {"id": "go_checkout", "enabled": true,
     "input": {"type": "object", "properties": {}}},
    {"id": "set_address", "enabled": false,
     "input": {"type": "object",
               "properties": {"address": {"type": "string"}},
               "required": ["address"]}},
    {"id": "pay", "enabled": false,
     "input": {"type": "object", "properties": {}}}
  ]
}

```

Hold on to this example, because it is exactly the state where the one failure of the view document (C4) happens. The correct next move is `go_checkout`. In the model run, the model instead called `set_address` from this state, with `enabled: false` printed in front of it,

and the backend rejected the call. The document told the truth and the model did not read it. Section 6.3 quantifies that failure.

Two properties make the document trustworthy as a measurement instrument. First, it is generated from backend state, so it is faithful by construction, and the enable rules mirror the ones the human page already uses: `add_to_cart` only when a valid size is selected, `set_address` only at checkout, `pay` only at checkout with a non-empty cart and address, and the size list always equal to what is actually in stock. Second, an automated consistency check forbids the document from ever advertising an action the backend would reject, so a good result for the view document (C4) cannot be an artifact of a surface that quietly disagrees with the application. Because it is written by hand against a known backend, it should be read as an upper bound on how good such a surface can be.

5. What we measure

Per run. Success (for refusal tasks, correctly declining). Input and output tokens summed over steps. Step count. Illegal actions (backend rejections). Malformed actions (replies that are not a usable action at all), counted separately so that formatting failures are never disguised as validity failures.

How tokens decompose. Each step costs a roughly constant overhead (instructions, task, prior actions) plus the current observation, whose size depends on the condition. A representation can therefore win in two ways: by being lighter per look, or by needing fewer looks.

Text conditions. For the accessibility tree (C2), flat tools (C3), and view document (C4), the observation is serialized text, and its cost is simply the length of that text under a fixed tokenizer. Two structural facts matter. Flat tools (C3) re-sends its full function schema with every call, because that is how the native tool-calling interface works, and the schema dominates its per-step cost. The view document (C4) pays for the document itself. Median per-step input along the shortest correct path:

Condition	System	Task and prior	Tool schema	Observation	Per-step total
Screenshot (C1), estimate	186	62	not sent	1105 (image)	1353
Accessibility tree (C2), measured	188	62	not sent	~650 (tree)	~900
Flat tools (C3)	159	62	490	25	736
View document (C4)	188	62	not sent	422	672

The flat tools (C3) schema (490 tokens) dwarfs its tiny status reply (25 tokens). The view document (C4) spends its budget on the document itself (422 tokens). The accessibility tree (C2) is measured from live browser traces because it requires a rendered page. The other three rows are reconstructed deterministically with no model call.

The screenshot (C1) is priced differently. Images are billed by resolution tiles, not by tokenized content. Under the widely documented gpt-4o-style rule (fit within a 2048 by 2048 box, scale the short side toward 768 pixels, cover with 512-pixel tiles, charge a base plus a per-tile rate), our 1280 by 800 viewport costs six tiles, about 1105 image tokens, and a busier page at the same size costs no more. Providers also report image cost differently. OpenAI itemizes image tokens. Vertex and Gemini fold them into the total prompt count, which is why our screenshot (C1) runs report an itemized image count of zero alongside roughly 46,000 input tokens per task. The image is billed either way, so we read the screenshot (C1) cost from total input tokens. Tiling constants vary by provider, and we present these numbers as a labeled estimate rather than a universal formula.

Analysis. We report one table per model and condition, and a cost-reliability frontier of median input tokens against success rate. Because every condition runs the same tasks, we also compare the view document (C4) against flat tools (C3) *paired per task*: average each task over its repeats, take the difference, and build a 95 percent confidence interval over tasks by bootstrap. A bootstrap resamples the per-task differences with replacement many times, recomputes the mean each time, and reads the interval off the middle 95 percent of those means; it assumes nothing about the shape of the data, which matters with only seven purchase tasks. Success and refusal tasks are reported separately. Models are never pooled.

Latency is not yet instrumented. Time weights round trips differently from tokens, so a timed re-run could shift the structured comparison without changing any token number. It is listed as future work.

6. Results

All results in this section come from one model on one store. The finding to trust most is the coarse ordering (human surfaces against structured ones). The fine margin between flat tools (C3) and view document (C4) is a provisional, single-model result.

6.1 What each representation costs before any model touches it

Replaying each task's shortest correct path and counting observation tokens with a fixed tokenizer gives a model-free baseline. One look costs a median of roughly 1105 tokens on the screenshot (C1), 512 on flat tools (C3) (the re-sent schema), and 453 on the view document (C4). Text is about 2.4 times cheaper per look than pixels, and the view document (C4) is

slightly cheaper per look than flat tools (C3). This measures the observation term only. Real runs add step-count effects, and Section 6.3 shows they reverse the ordering of flat tools (C3) and view document (C4). Section 7.1 reports how these per-look costs move as the catalog grows.

6.2 The apparatus, checked without a model

The deterministic oracle described in Section 4 replays the known-correct policy and scores 100 percent on flat tools (C3) and view document (C4). Its token counts are locally tokenized rather than provider-billed, and the flat tools (C3) schema is not included in that count because the oracle loop supplies the schema outside the counted text. The screenshot (C1) and accessibility tree (C2) are out of the oracle's scope, since they would need pixel-level or element-level oracles. The point of the control is attribution: when a model later misses, the miss belongs to the model.

6.3 The model run: **gemini-2.5-flash**, five repeats

One general model, temperature zero, step cap twenty, four conditions, ten tasks, five repeats: 200 runs. Confidence intervals are bootstrap intervals over the fifty runs (ten tasks times five repeats) in each condition.

Table 1. Per-condition results.

Condition	Success (95% CI)	Median input tokens (95% CI)	Median steps	Illegal/step	Malformed/step
Screenshot (C1)	30% [18, 42]	~46,338 [46,324, 46,368]	20	0.000	0.000
Accessibility tree (C2)	72% [60, 84]	~6,770 [5,835, 9,714]	7	0.198	0.116
Flat tools (C3)	100% [100, 100]	~3,134 [3,118, 3,154]	7	0.000	0.000
View document (C4)	92% [84, 98]	~4,455 [4,413, 4,491]	6	0.016	0.000

The whole sweep cost about 2.82 million input tokens and 35 thousand output tokens, about \$0.93 at the recorded rates (\$0.30 per million input tokens, \$2.50 per million output tokens). The screenshot (C1) alone is about 73 percent of that bill (2.11 million input; the accessibility tree (C2) used 385 thousand, flat tools (C3) 138 thousand, view document (C4) 193 thousand). Screenshots dominate the cost before the question of whether they succeed is even asked.

Success. The screenshot (C1) passes 30 percent of runs and *none* of the seven purchase tasks; its passes are all refusals. The accessibility tree (C2) reaches 72 percent but earns it noisily, mixing backend rejections with malformed replies. Flat tools (C3) is perfect. The view document (C4) is near-perfect at 92 percent, and its entire deficit is one task, t10. With five repeats over ten tasks these intervals are wide, and the structured gap rests on that single task.

Tokens. The screenshot (C1) costs about ten times the view document (C4) and about fifteen times flat tools (C3) per task. Among the text conditions, flat tools (C3) is cheapest, the view document (C4) is next, and the accessibility tree (C2) is the most expensive and the most variable, because tree size tracks page content. Note the reversal against Section 6.1: per look, the view document (C4) is cheaper than flat tools (C3), but in real runs the document grows as the cart fills, and the tiny status replies of flat tools (C3) win on total cost.

Steps. The view document (C4) finishes fastest, median six steps, against seven for flat tools (C3) and the accessibility tree (C2). The screenshot (C1) sits at the cap of twenty because it never completes a purchase and simply spends its budget.

Errors. The screenshot (C1) and flat tools (C3) commit zero illegal and zero malformed actions. The view document (C4) commits 0.016 illegal actions per step, all on t10. The accessibility tree (C2) is the outlier, 0.198 illegal and 0.116 malformed per step: the human tree invites named clicks the backend rejects, and the model periodically produces replies the loop cannot parse at all. One caution: the screenshot (C1)'s clean error rate is not a virtue. A coordinate click rarely trips the backend's validity check, so its failures show up as capped steps and spent tokens instead.

Table 2. Per-task readout. Each cell is success rate, median steps, and median input tokens over five repeats. Refusal tasks are marked *r*.

Task		Screenshot (C1)	Accessibility tree (C2)	Flat tools (C3)	View document (C4)
t01		0% · 20 · 46,368	20% · 12 · 11,702	100% · 7 · 3,154	100% · 6 · 4,498
t02		0% · 20 · 46,368	100% · 11 · 10,809	100% · 7 · 3,154	100% · 6 · 4,486
t03	<i>r</i>	100% · 2 · 4,184	100% · 2 · 1,997	100% · 2 · 1,081	100% · 2 · 1,597
t04		0% · 20 · 46,268	0% · 6 · 5,835	100% · 7 · 3,118	100% · 6 · 4,423
t05	<i>r</i>	100% · 20 · 46,018	100% · 3 · 2,605	100% · 2 · 713	100% · 2 · 1,528
t06		0% · 20 · 46,328	100% · 10 · 9,714	100% · 7 · 3,144	100% · 6 · 4,491

Task		Screenshot (C1)	Accessibility tree (C2)	Flat tools (C3)	View document (C4)
t07	<i>r</i>	100% · 20 · 46,348	100% · 2 · 1,990	100% · 2 · 1,079	100% · 1 · 905
t08		0% · 20 · 46,324	100% · 7 · 6,690	100% · 7 · 3,124	100% · 6 · 4,413
t09		0% · 20 · 46,428	0% · 20 · 21,658	100% · 12 · 5,833	100% · 10 · 7,867
t10		0% · 20 · 46,368	100% · 7 · 6,770	100% · 7 · 3,154	20% · 6 · 4,544

The table makes four patterns visible. The screenshot (C1) hits the step cap on every task it does not refuse, at a nearly flat 46 thousand tokens, because image cost is set by the viewport. The accessibility tree (C2) is erratic, failing t01 and t04 outright and capping out on the two-item task t09 with the heaviest text-condition token bill (about 21.7 thousand). Flat tools (C3) is uniformly clean. The view document (C4) matches it everywhere except t10, usually one step faster and always at more token cost.

The paired comparison. Averaging each task over its five repeats and taking the difference (view document (C4) minus flat tools (C3)) gives a per-task comparison with a bootstrap interval over tasks.

Table 3. Paired per-task differences. Positive means the view document (C4) is higher.

Task set	n	Metric	Mean difference	95% CI	Excludes 0?
success	7	success	−0.114	[−0.343, 0.000]	no
success	7	input tokens	+1,404	[+1,317, +1,549]	yes
success	7	steps	−1.17	[−1.51, −1.00]	yes
refusal	3	success	0.000	[0.000, 0.000]	no
refusal	3	input tokens	+386	[−174, +815]	no
refusal	3	steps	−0.33	[−1.00, 0.00]	no

On purchase tasks, the view document (C4) saves steps (the interval excludes zero, which is precisely what a current-view document is for) and costs more tokens (the interval excludes zero, and all seven tasks point the same way). It does not measurably change success (the interval includes zero, and the entire point estimate is t10). So on this store and this model, flat tools (C3) is preferred on the frontier: cheaper, and at least as reliable. The strong form of H2,

that the view document (C4) dominates, is not supported here. On refusal tasks nothing separates the two.

The one failure, told plainly. Task t10 asks the agent to buy a Navy Crew Tee in size L and ship it to 77 Oak Lane, Denver. The legal order of operations is: open the product, add it to the cart, go to checkout, and only then set the address. The model instead tried to set the address while still on the product page. That is exactly the state printed in Section 4.1, where `set_address` is marked `enabled: false`. The backend rejected the call, and the failing runs never went on to place the correct order. The document was correct, the backend agreed with it, and an automated check pins that agreement, so this is not an interface bug. It is a model ignoring a rule the interface stated. The failure reproduced on four of five sweep repeats and five of five in a follow-up diagnostic, always as the same premature call. One harness detail is worth knowing when reading this: in the view document (C4) condition the loop does not echo the backend's error message. After a rejected call the model simply sees the fresh document again, with the action still marked disabled. The first premature call is therefore a clean case of ignoring the flag; what happens after it is shaped partly by that feedback choice, which Section 9 lists among the limitations. The lesson generalizes: a truthful document cannot help a model that does not read `enabled`. How often a model attempts actions the interface marked unavailable (its ignored-affordance rate) is a quantity the next model must measure.

6.4 A second model (not yet run)

Whether flat tools (C3) stays preferred is an open question about models, not about the store. The single most informative datum from a second general model is whether it respects `enabled: false` on t10.

6.5 An ablation (not yet run)

Stripping the `enabled` flags and argument constraints out of the view document (C4), while keeping the view and entities, would separate the document's two entangled mechanisms: compactness and constraint-carrying. If the step savings vanish, the constraints were doing the work.

7. Discussion

7.1 What these results establish

- **Representation moves every measured quantity (RQ1).** Success from 30 to 100 percent, median tokens by a factor of about fifteen, steps from six to the cap, and error rates from zero to 0.198 illegal per step, all from re-describing the same store to the same model.

- **Human surfaces are dominated here (RQ2, first half).** Both the screenshot (C1) and the accessibility tree (C2) are more expensive and less reliable than either structured condition.
- **The document does not yet beat the catalog (RQ2, second half).** Flat tools (C3) is preferred over the view document (C4) on tokens and success, while the view document (C4) reliably saves steps. The mechanism a purpose-built surface promises is visible, and it is not yet large enough to pay its token bill on this store.
- **The one failure is behavioral, not structural.** On t10 the document told the truth and the model ignored it. That distinction only exists because the surface is checked against the backend, which is an argument for building agent surfaces that can be checked. One reading these data cannot confirm but that fits them: models tuned for tool calling may treat any listed action as callable and give an `enabled` flag no special weight.

It is worth saying why this store is the hard case for the view document (C4). MiniShop is tiny: few views, short tasks, little state to track. That is exactly where the weakness of flat tools (C3), making the model carry the state in its head, costs the least. A representation that pays per step to describe state earns the least where there is the least state to describe.

The store is small in a second way, and this one cuts against both structured conditions equally. With eight products, the document can afford to embed the whole catalog on the catalog view and a complete list of product ids inside its action schemas, and flat tools (C3) can afford to return the whole catalog from a single lookup. We measured what happens to that arrangement as the catalog grows, with no model call: the same canonical task paths are replayed against the real catalog padded with synthetic products, and observation tokens are counted exactly as in Section 6.1, whose numbers the eight-product row reproduces. Median observation tokens per step:

Catalog products	Screenshot (C1), estimate	Flat tools (C3)	View document (C4)
8	1105	512	453
50	1105	512	992
500	1105	512	6,767
5,000	1105	512	64,517

The view document (C4) overtakes flat tools (C3) at a catalog of thirteen products. Its per-look advantage in Section 6.1 is therefore a small-catalog artifact of this particular document design, which ships the whole catalog and a complete id list in every look. The flat row for flat tools (C3) deserves its own caution: it stays flat only because its canonical path never reads the catalog. A real flat tools (C3) run calls `list_products` at least once per task, and that one reply grows the same way, from 369 tokens at eight products to 294,897 at five thousand. So at

scale neither condition survives as designed: the view document (C4) pays per look, and flat tools (C3) pays per lookup. The screenshot (C1) column is flat because image cost is set by the viewport, not by content. The design conclusion stands, and it is now measured rather than predicted: a scale-aware document must grow with what is on the current screen (paginated entities, and id references instead of a complete id list), not with the size of the database. None of this changes any recorded model result. Tables 1 through 3 are eight-product numbers and stand as reported.

7.2 What later measurements must show

These are predictions, not results.

1. **A second general model (RQ3).** If it obeys `enabled: false` on t10, the view document (C4) closes the success gap. If it also ignores the flag, the miss is a property of current models rather than of one model. Either answer is informative. Models stay unpooled, and if a model lacks vision, the screenshot (C1) is dropped and reported as such.
2. **The ablation.** Decides whether the document's value is compactness or constraints.
3. **Latency.** The view document (C4) uses fewer round trips. Priced in time instead of tokens, the verdict could flip without any number in Table 1 changing.
4. **A harder application.** More views, more state, more ways to act illegally. The natural choice is a single-page application, where the server cannot even name the current view; Section 8.2 explains why that is the sharpest test. That is where we expect the state-tracking tax on flat tools (C3) to grow, and where the trade should tilt. Until that measurement exists, "flat tools (C3) wins on MiniShop" must not be read as "do not build agent surfaces."

8. How an application could ship a view document (C4)

The document we measured was written by hand, so it represents the best case. The encouraging fact for adoption is that almost everything in it is validity logic the application already has. A human page already disables Add to cart until a size is chosen, already validates the address field, already hides Pay until the cart and address are set. Shipping a view document (C4) is mostly *writing down checks the application already performs*, not authoring new business rules. The backend remains the authority, and the document must never advertise an action the backend would reject, which is a testable property. This section explains how the page and the document stay consistent in this study, what that mechanism assumes about an application, and the realistic routes by which an application, new or existing, could produce such a document.

8.1 How the page and the document stay in sync

The phrase "a JSON version of the website" suggests an export: some job that walks the site, writes a copy, and must then be kept in step with the live pages. Nothing of that kind exists in this study, and the distinction is the whole point. MiniShop holds one session record on the server: the current view, the open product, the selected size, the cart, the address, the orders. When a person requests a page, an HTML template renders that record. When an agent requests the document, a second renderer serializes the same record. Both are produced at request time from the same source. They can no more drift apart than two templates of the same page can, because neither is derived from the other; both are projections of one state. The automated consistency check described in Section 4.1 covers the remaining risk, that the document's `enabled` flags might disagree with what the backend enforces. Synchronization is therefore not an ongoing maintenance task in this architecture. It is a property of generating both surfaces from one state at the moment of each request.

Two caveats keep this honest. First, the document is true at the moment it is read, not at the moment the agent acts. Stock can sell out and another session can change the world between the look and the click, so `enabled: true` is current advice, not a reservation. The correct reading is "worth attempting now," never "guaranteed to succeed," and the backend must still validate every action when it arrives, which here it does. MiniShop's tasks have a single actor, so this gap never shows in the tables; on a live site it would. Second, the document describes one session, and it is only meaningful to the agent acting in that session: the same login, the same cart, the same prices that user would see. The harness passes an explicit session id with every request; a production version would ride the same cookies and authentication as the page it accompanies.

8.2 What this mechanism assumes about the application

Generating the document from backend state quietly assumes the backend knows what the user is looking at. In MiniShop it does. The current view is a field in the server session, which is how server-rendered, multi-page applications work. A large share of the modern web does not work that way. In a single-page application, where the browser runs the interface and calls the server only for data, the state the document needs most (which screen is open, what is selected, what has been typed but not yet saved) lives in the client, and the server cannot name the screen at all. For such an application a faithful view document (C4) cannot come from the server alone. It would have to be produced where the view state lives, for example by serializing the application's own client-side state store, and nothing in this paper measures that variant. This is the strongest assumption the study rests on, so it is stated plainly: the measured result covers applications whose server holds the view state. The second application planned in Section 10 should be a single-page application precisely because it attacks this assumption.

8.3 Five ways to produce the document

From highest leverage to most general:

1. **Framework-emitted.** The developer declares views, state, and actions once, and the framework renders both the human page and the view document (C4) from that single declaration. Near-zero marginal effort and no drift between the two surfaces, but it requires framework support that is not yet common.
2. **Projected from a spec.** Applications with a GraphQL schema, form validation schemas, or server-driven UI already hold their capabilities in machine-readable form. Types become entities, mutations become affordances, validation rules become `enabled` logic. Fidelity is high wherever the spec is the real authority.
3. **Hand-written per view (this study).** Faithful by construction and pinned by tests, but bespoke per view, so it suits a few high-value flows rather than an ecosystem. As a size reference, MiniShop's document generator is roughly three to four times the code of its flat tools (C3) catalog, and it mirrors checks the human page already makes.
4. **Compiled from the human page.** A compiler reads the DOM, ARIA roles, and disabled attributes of the rendered page, at runtime in a browser, an extension, or a rendering proxy, and emits a document. It requires nothing from the application, but it pays the page rendering it hoped to avoid, it inherits only what the page happens to render, and it can disagree with the backend, so it must be validated before it is trusted.
5. **Model-extracted.** A model reads the screen and writes the document. It works on any application with zero cooperation, and it inverts the economics: the cost returns on every step, along with the risk of hallucinated actions. A fallback, not the goal.

8.4 Getting there from an existing application

An existing site does not have to adopt this everywhere or all at once, and the incremental route follows from the per-view design. Pick the one or two flows agents actually attempt (checkout, booking, an account change), write or project the document for those views only, and leave the rest of the site untouched, retiring the gap one view at a time. The per-view cost is the one quantified in path 3 above. Which route fits depends on where the truth lives. A server-rendered application is the easy case measured here: the work is a second serializer over session state the server already holds. A single-page application keeps the truth in the client, so the serializer belongs there (Section 8.2). An application that cannot cooperate at all is left with the compiled path and its fidelity costs. Deploy drift is the remaining operational risk on every route except the framework-emitted one: the page changes every release and a hand-written or compiled document ages with it, so the consistency check belongs in the release tests, where silent drift becomes a failing build instead of a lying surface.

The underlying trade. A view document (C4) pays once, at authoring or compile time, for the ability to state what is true and allowed at every step. Flat tools (C3) pays forever at runtime, because the model must re-derive state from its own history on every step, for the life of the deployment. On a tiny store that runtime tax is small, which is exactly what Section 6 measures. It grows with views, state, and ways to be wrong, and one authored document serves every agent that ever visits the application.

A short recipe for a developer today. Enumerate the views. For each view, expose the state fields, the entities, and the actions, reusing the exact enable-conditions and validators the human page already has. Serve the document alongside the page. Keep backend enforcement in place. Add one automated check that the document never advertises what the backend would reject.

Table 4. The five paths at a glance.

Path	Application effort	Coverage	Fidelity	Best fit
Framework-emitted	Low at scale	High	High, no drift	New applications on agent-aware frameworks
Projected from a spec	Low to medium	Wherever the spec rules	High there	GraphQL, form schemas, server-driven UI
Hand-written per view	Medium per app	What is authored	High, test-pinned	A few high-value flows; this paper
Compiled from the human page	Near zero	Any app with a page	Lossy, must be checked	Legacy applications
Model-extracted	Near zero	Universal	Lowest, cost recurs	Fallback only

Measuring the two generated paths (compiled and model-extracted) against the hand-written upper bound established here is future work.

9. Limitations

Scope. One model, one small synthetic store of eight products (Section 7.1 measures how observation costs scale when the catalog grows), ten tasks, and one application architecture, a server that holds all view state (Section 8.2). The paired intervals resample over seven purchase tasks and three refusals, so they are wide by construction. Nothing here generalizes beyond a controlled shopping task yet.

The document is an upper bound. A hand-written view document (C4) is as good as such a surface gets. A sloppier generated one would do worse, and by how much is unmeasured.

Entangled mechanisms. Compactness and constraint-carrying are not separated until the ablation runs.

Error feedback differs between the structured conditions. Flat tools (C3) receives the backend's reply to each call, including any error message, as its next observation. The view document (C4) receives only the fresh document, so a rejected call shows up as unchanged state rather than as an error message. In this sweep the asymmetry had little room to matter, because flat tools (C3) committed no illegal actions at all, but a re-run that echoes error text into the view document (C4) condition is cheap and could plausibly recover t10.

Refusals can pass by inaction. They are therefore reported apart from purchases throughout.

The grader checks the minimum. It reads the most recent order, requires the requested item to be present rather than the order to be exact, matches the street without the city, and scores a refusal as the absence of the forbidden order (Section 4). A stricter grader could only lower success numbers, never raise them. The step counts argue the recorded numbers would survive it: passing structured runs used the shortest correct path, and refusal passes were too short to contain a substitute purchase. The accessibility tree (C2) passes are long enough that untidy orders cannot be fully ruled out without re-checking, which Section 10 queues.

Purchase runs end at the grader's signal. The loop checks the grader after every step and stops a purchase run the moment it passes, so the step medians never include the agent recognizing completion, and anything an agent might do after succeeding is unobserved. Refusal runs have no such signal and end only when the model stops or the cap is hit.

Image accounting is provider-specific. Some providers itemize image tokens and some fold them into the prompt count, so the screenshot (C1) is read from total input tokens, and the tiling numbers are a labeled estimate.

Temperature zero is not determinism. Results are averaged over five repeats. Flat tools (C3) was stable across repeats, the view document (C4) varied only on t10, and the accessibility tree (C2) was noisy throughout.

The memory policy is fixed. Agents carry only the task, their prior actions (the actions themselves, not their results), and the current observation. This is the same for every condition, but it also means flat tools (C3) cannot compensate by accumulating a long transcript, and other memory policies are untested.

Each prompt coaches its own condition. Every condition's system prompt adds a few lines of usage guidance that cannot be identical across conditions (Section 4), so part of any between-

condition gap may be instruction authorship rather than representation. Equalizing the guidance is queued in Section 10.

The screenshot (C1) error rate understates its errors. Coordinate clicks rarely trigger the backend's validity check, so for the screenshot (C1) the informative numbers are success, steps, and tokens.

Wide intervals are not equality. An interval that includes zero, as the paired success difference does, means the data cannot distinguish the conditions at this sample size, not that they are the same.

Specialized models are out of scope. The main line uses general models that can run all four conditions through one API. Pixel-native computer-use models are a planned final check, and the second general model has not yet been run.

10. Future work

Nearest first: tighten the grader to exact-order matching, full addresses, and substitution-proof refusals, then re-check the sweep under it; equalize the usage guidance in the condition prompts and re-run the structured pair; instrument latency and re-run timed; run a second and then a third general model, reporting each separately; run the constraint-stripping ablation; re-run the view document (C4) with backend error messages echoed into the next observation, to test whether t10 recovers; report an ignored-affordance rate per model. After that: a second, more complex application, ideally a single-page application whose view state lives in the client (Section 8.2), which is the main lever on external validity and the sharpest test of this study's strongest assumption; a validated Agent Experience metric suite, which is a separate paper; and automatic generation of the agent surface, compiled from the page or extracted by a model, measured against the hand-written upper bound established here.

Appendix: reproducing the numbers

The store, the four-condition harness, the grader, and every trace behind the tables live in the project repository, with no hidden state. The sequence is fixed. A scripted non-model policy must first complete every purchase task on flat tools (C3) and view document (C4) and be refused on the refusals. The model-free baseline of Section 6.1 then runs with a fixed tokenizer and no API call. The oracle of Section 6.2 validates the pipeline. The model sweep writes its 200 traces, and the report layer aggregates them into Tables 1 through 3 with their bootstrap intervals. Dependencies and the browser used by the screenshot (C1) and accessibility tree (C2) conditions are pinned in the repository, and this manuscript rebuilds to HTML and PDF from its Markdown source.

References

Verified sources only. Each entry was checked against the primary paper or official documentation. No other citations are asserted.

- WebArena: A Realistic Web Environment for Building Autonomous Agents.
<https://arxiv.org/abs/2307.13854>
- VisualWebArena: Evaluating Multimodal Agents on Realistic Visually Grounded Web Tasks. <https://aclanthology.org/2024.acl-long.50/>
- Mind2Web: Towards a Generalist Agent for the Web. <https://arxiv.org/abs/2306.06070>
- WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents.
https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf
- AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents.
<https://arxiv.org/abs/2405.14573>
- MiniWoB++ (Reinforcement Learning on Web Interfaces using Workflow-Guided Exploration). <https://arxiv.org/abs/1802.08802> ; docs: <https://miniwob.farama.org/>
- Anthropic: Introducing computer use (Claude 3.5 Sonnet).
<https://www.anthropic.com/news/3-5-models-and-computer-use>
- UI-TARS: Pioneering Automated GUI Interaction with Native Agents.
<https://arxiv.org/abs/2501.12326>
- SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents.
<https://aclanthology.org/2024.acl-long.505/>
- UIFormer / From User Interface to Agent Interface: Efficiency Optimization of UI Representations for LLM Agents. <https://arxiv.org/abs/2512.13438>
- Prune4Web: DOM Tree Pruning Programming for Web Agent.
<https://ojs.aaai.org/index.php/AAAI/article/download/40772/44733>
- A2UI (Agent-to-UI) Protocol. <https://a2ui.org/>
- Model Context Protocol (MCP). <https://modelcontextprotocol.io/>